Constants. Rule Float-Const assigns a constant c the type $\mathbf{float}[B; V]$ whenever has(V, c) holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in V ensures later comparisons know which values must remain distinguishable.

Continuous distributions. Rule Continuous-1 types a single-argument distribution when its argument e has type $\mathbf{float}[B; V]$ and recovers(B, V):

Definition 3.2. Let $B \in \mathcal{B}$ and $V \in \mathcal{V}$. We say that recovers(B, V) if $B = \top$ or ($V \neq \top$ and every interval I induced by B contains at most one value in V, i.e., $\forall I \in \text{Intervals}(B) : |V \cap I| \leq 1$).

Intuitively, recovers(B, V) means that the discretization induced by B is lossless on V: from the resulting interval index, the original value in V can be uniquely recovered. For example, $\mathbf{exponential}(e)$ can be discretized only by finite dispatch when e takes finitely many a finite, nonempty set of recoverable values. Thus V must be finite and The cut set B must separate those values so the discretized program can determine which one occurred. §5 revisits this invariantDistinguishability alone permits $V = \emptyset$; nonemptiness is a separate prerequisite of the finite-dispatch transformation. The additional premise $B_1 = \top \iff B = \top$ ensures that the distribution and its parameter use the same representation: either both remain real-valued or both are discretized. §5 revisits these invariants. The rule for two-argument distributions is analogous and requires the same representation agreement for both parameters.

Comparisons. For $e < c$ with $e : \mathbf{float}[B_1; \top]$ and $c : \mathbf{float}[B_2; \{c\}]$, the discretized e must still decide the predicate, so B_1 must include $<c$. If c can take any value in V_2, every $<v$ for $v \in V_2$ must appear. Symmetrically, for $c < e$ with $e : \mathbf{float}[B_2; V_2]$, B_2 must contain $\leq c$ so the discretized intervals know on which side of c they fall. For example, $2.3 < e$ requires cuts separating $(-\infty, 2.3]$ from $(2.3, \infty)$; finer partitions are allowed but not required.

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the answers$_<(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that answers$_<(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ <x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ \leq x \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates.* That is, if B contains $<x$ then the discretized value of $e : \mathbf{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \mathbf{float}[B; V_1]$ and $e_2 : \mathbf{float}[B; V_2]$, then answers$_<(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all $<x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that answers$_\leq(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ \leq x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ <x \in B$.

coincides with the probability of the continuous expression evaluating to 10 (resp. 20). Let us (partially) apply $\mathcal{D}[\![e : \textbf{bool}]\!]$, resulting in:

```
let x = if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3 in
( let v1' = D[[0 : float[{<=0}; {0}]]] in
    let v2'  = D[[x : float[{<=10,<=20}; {10,20}]]] in
    if v1' ==#2 D[[0 : float[{<=0}; {0}]]] &&
                v2' ==#3 D[[10 : float[{<=10,<=20}; {10,20}]]]
    then D[[uniform(0,10) : float[{<5}; T]]]
    else D[[uniform(0,20) : float[{<5}; T]]] ) <#2 D[[5 : float[{<5}; {5}]]]
```

The continuous $\textbf{uniform}(0, x)$-expression is replaced by a case distinction on the discretizations of $\textbf{uniform}(0, 10)$ and $\textbf{uniform}(0, 20)$, with each case executed with the correct probability 0.5. Since this reduces the remaining discretization process to discretizing expressions with *constant parameters only*, the remaining steps are completely analogous to Theorem 5.1.

Crucially, the *recoverability* constraints in the Continuous-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule Less (resp. Less-Equal) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2) : \textbf{bool}$, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
        uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
        then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved The following proposition isolates the representation property established by the continuous-distribution premises; its proof is in §E.1: . Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.

Proposition 5.4 (Continuous-distribution representation agreement). *Let $d(e_1, \ldots, e_r)$: `float`$[B; \top]$ be typed by Continuous-1 or Continuous-2. If $B = \top$, representation agreement makes every parameter cut set $\top$, so the retained sampler and its parameters remain real-valued. If $B \neq \top$, representation agreement makes every parameter cut set finite. When every parameter value set is also finite and nonempty, the generated call-by-value*

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \texttt{fin}(|B|+1) & \text{if } \tau = \texttt{float}[B;V] \text{ and } B \neq \top \\ \texttt{float}[B;V] & \text{if } \tau = \texttt{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

dispatch term has result type $\texttt{fin}(|B|+1)$*, provided the recursively transformed parameters and the emitted target let-bindings, finite equalities, and conditionals have their stated typings.*

This proposition closes the source/target representation mismatch for continuous distributions. It is not a proof of universal type preservation for $\mathcal{D}[\![\cdot]\!]$. Such a theorem additionally requires separate results for subtyping, nonempty dispatch, the generated target syntax, and the semantic coupling used by the soundness argument.

6 Operational Semantics and Soundness

We now establish the soundness of our approach in the following sense: For *every* expression $e : \texttt{bool}$, possibly including *continuous sampling, higher-order functions, recursion, and conditioning,*

$$e : \texttt{bool} \textit{ and } \mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool} \textit{ (obtained from Figure 13) are semantically equivalent.}$$

The first step is to make formal sense of semantic equivalence. For that, we will define a monadic operational semantics for our language in §6.1. We will then proceed by proving the above claim. This is non-trivial: discretization decisions are driven by non-local, type-propagated information, so we cannot proceed by a local rule-by-rule argument. To prove the claim rigorously, we relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle : \mathit{Expr}_\tau \to \mathcal{D}(\mathit{Vals}_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \texttt{bool}\rangle = \langle \mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!] : \mathit{Expr}_\tau \to \mathcal{D}(\mathit{Expr}_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \texttt{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\texttt{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; y : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 1 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 8) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a discretized gaussian,
    then use the resulting value to update
    flags *)
let step = fun ys ->
   update (discrete(0.5, 0.0199388,
                    0.0748961, 0.0494739,
                    0.0747339, 0.0632619,
                    0.0309625, 0.0280777,
                    0.158655)) ys in

 (* Number of samples determined by
    unbounded recursion *)
 let cover = fix cover st :=
   if discrete(0.1, 0.9) < 1 then st
   else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 20. A *discretized* infinite stochastic process for random interval covering over $[0, 1]$, involving continuous sampling, higher-order recursion, and lists.

B.2 Bloom Filter

The full program for the case study from §7.1.3 is shown in Figure 23. The discretized program is shown in Figure 24.

C List of Supported Slice Distributions

The list of distributions supported by Slice, along with their probability density / mass functions, is given in the following table. Slice supports discrete distributions by applying the same cut-set and value-set analysis used for continuous distributions, *provided that the discrete distribution admits a well-defined CDF*. The extension to discrete distributions does not introduce any fundamental changes to the discretization algorithm.

D Typing Rules for Full Slice Language

The typing rules for general language constructs is given in Figure 25.

E Discretization Proofs

E.1 Proof of LemmaProposition ??5.4

Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.

Proof. We proceed by induction on the structure of the typing derivation $\Gamma \vdash e : \tau$, matching each case of the discretization function $\mathcal{D}[\![\cdot]\!]$ defined in Figure 13. give the two-argument case; the one-argument case is the

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
   respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Ensure sampled y hits at most one
   interval *)
let non_overlap =
  fix non_overlap y := fun seen -> fun ys ->
    match ys with
      | nil -> true
      | h :: t ->
          let hit = (snd h) y in
          if hit && seen then false
          else non_overlap y (seen || hit) t
    end in

(* Use the sampled value to update flags *)
let step = fun y -> fun ys ->
  update y ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else
    let y = gaussian(0,1) in
    let _ = observe(non_overlap y false st) in
    cover (step y st) in

(* Have all intervals been covered
   at least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run the process and query whether
   the whole interval was covered *)
let final = cover init in
  is_covered final
```

Fig. 21. An infinite stochastic process for random interval covering over $[0, 1]$, involving conditioning, continuous sampling, higher-order recursion, and lists.

same with one parameter and one dispatch dimension. Let

$$e = \mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1], e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B; \top]$$

be typed by Continuous-2. Its premises include $B_i = \top \iff B = \top$ for $i \in \{1, 2\}$.

Case $e = x$. Since $\Gamma(x) = \tau$, we have $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\tau]\!]$. Moreover, $\mathcal{D}[\![x : \tau]\!] = x$, so $\mathcal{D}[\![\Gamma]\!] \vdash x : \mathcal{D}[\![\tau]\!]$.

Case $e = (c : \mathbf{float}[B; V])$. The Float-Const rule was applied, so has(V, c) holds. If $B = \top$: then $\mathcal{D}[\![c : \mathbf{float}[B; V]]\!] = c$ and $\mathcal{D}[\![\mathbf{float}[B; V]]\!] = \mathbf{float}$. Since has$(V, c)$ still holds, c is well-typed at $\mathbf{float}$ by Float-Const. If $B \neq \top$: then $\mathcal{D}[\![c : \mathbf{float}[B; V]]\!] = k_{\#|B|+1}$ for the unique k such that $c \in \text{Intervals}(B)_k$, and $\mathcal{D}[\![\mathbf{float}[B; V]]\!] = \mathbf{fin}(|B| + 1)$. Since $0 \leq k < |B| + 1$, the finite constant $k_{\#|B|+1}$ is well-typed at $\mathbf{fin}(|B| + 1)$ by FinConst.

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
   respective (possibly overlapping) bounds *)
let init =
  (false, within 1 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 8) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Ensure sampled y hits at most one
   interval *)
let non_overlap =
  fix non_overlap y := fun seen -> fun ys ->
    match ys with
      | nil -> true
      | h :: t ->
          let hit = (snd h) y in
          if hit && seen then false
          else non_overlap y (seen || hit) t
    end in

(* Use the sampled value to update flags *)
let step = fun y -> fun ys ->
  update y ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if discrete(0.1, 0.9) < 1 then st
  else
    let y = discrete(0.5, 0.0199388,
                     0.0748961, 0.0494739,
                     0.0747339, 0.0632619,
                     0.0309625, 0.0280777,
                     0.158655) in
    let _ = observe(non_overlap y false st) in
    cover (step y st) in

(* Have all intervals been covered
   at least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run the process and query whether
   the whole interval was covered *)
let final = cover init in
  is_covered final
```

Fig. 22. A *discretized* infinite stochastic process for random interval covering over $[0, 1]$, involving conditioning, continuous sampling, higher-order recursion, and lists.

Case $e = (\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2)$. By the inductive hypothesis, $B = \top$, then $B_1 = B_2 = \top$. Assuming the recursive transformations of e_1 and e_2 have their transformed types, both remain float-typed. Hence the retained term

$$\texttt{uniform}(\mathcal{D}[\![e_1]\!], \mathcal{D}[\![e_2]\!])$$

Note that $\mathcal{D}[\![\Gamma, x : \tau_1]\!] = \mathcal{D}[\![\Gamma]\!], x : \mathcal{D}[\![\tau_1]\!]$. Applying Let gives uses the same real-valued representation for its parameters and result. This is precisely the mixed-representation case excluded by the new premises.

Case $e = (\texttt{if}\ e_1 : \texttt{bool}\ \texttt{then}\ e_2 : \tau\ \texttt{else}\ e_3 : \tau)$. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \tau_i]\!] : \mathcal{D}[\![\tau_i]\!]$ for $i = 1, 2, 3$, where $\tau_1 = \texttt{bool}$ and $\tau_2 = \tau_3 = \tau$. Since $\mathcal{D}[\![\texttt{bool}]\!] = \texttt{bool}$, applying If gives Now suppose $B \neq \top$,

```
(* Set bf[idx] := true, where idx is 0..3 and
   bf = (b0, (b1, (b2, b3))) *)
let set_true = fun bf -> fun idx ->
  let b0 = fst bf in
  let tail1 = snd bf in
  let b1 = fst tail1 in
  let tail2 = snd tail1 in
  let b2 = fst tail2 in
  let b3 = snd tail2 in
  if idx <= 0 then
    (true, (b1, (b2, b3)))
  else if idx <= 1 then
    (b0, (true, (b2, b3)))
  else if idx <= 2 then
    (b0, (b1, (true, b3)))
  else
    (b0, (b1, (b2, true))) in

(* Get bf[idx], where idx is 0..3 *)
let get = fun bf -> fun idx ->
  let b0 = fst bf in
  let tail1 = snd bf in
  let b1 = fst tail1 in
  let tail2 = snd tail1 in
  let b2 = fst tail2 in
  let b3 = snd tail2 in
  if idx <= 0 then b0
  else if idx <= 1 then b1
  else if idx <= 2 then b2
  else b3 in

(* Insert element q into bf using hash
   functions h1 and h2 *)
let insert = fun q -> fun h1
            -> fun h2 -> fun bf ->
  let index1 = h1 q in
  let index2 = h2 q in
  let bf1 = set_true bf index1 in
  let bf2 = set_true bf1 index2 in
  bf2 in

(* Search for q in bf using hash
   functions h1 and h2 *)
let search = fun q -> fun h1
            -> fun h2 -> fun bf ->
  let index1 = h1 q in
  let index2 = h2 q in
  let b01 = get bf index1 in
  let b02 = get bf index2 in
  b01 && b02 in

(* Hash functions *)
let h1 = fun q ->
  if q <= 0 then 0
  else if 0 < q && q <= 1 then 1
  else if 1 < q && q <= 2 then 2
  else 3 in

let h2 = fun q ->
  if q <= 0 then 3
  else if 0 < q && q <= 1 then 2
  else if 1 < q && q <= 2 then 1
  else 0 in

(* Test whether the Bloom filter is full *)
let full = fun bf ->
  let b0 = fst bf in
  let t1 = snd bf in
  let b1 = fst t1 in
  let t2 = snd t1 in
  let b2 = fst t2 in
  let b3 = snd t2 in
  b0 && b1 && b2 && b3 in

(* Initial, empty Bloom filter *)
let init = (false, (false, (false, false))) in

(* Query key *)
let key = discrete(0.0227501,
                   0.47725,
                   0.47725,
                   0.0227501) in

(* Insert discretized gaussian samples until
    bf is full or the key is reported
    present *)
let search_before_full = fix loop bf :=
  if full bf then bf
  else if search key h1 h2 bf then bf
  else
    let q = discrete(0.0227501, 0.47725,
                   0.47725, 0.0227501) in
    loop (insert q h1 h2 bf) in

(* Query whether search terminated before
   the filter became full *)
let bf_final = search_before_full init in
  not (full bf_final)
```

Fig. 24. A *discretized* 4-bit Bloom filter program with continuous samples, locality-sensitive hash functions, and search and insert functions.

Distribution	Density	Distribution	Density
Exponential	$\lambda e^{-\lambda x}$	Uniform	$\frac{1}{b-a}$
Laplace	$\frac{1}{2b} e^{-\frac{\lvert x-\mu\rvert}{b}}$	Gaussian	$\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
Cauchy	$\frac{1}{\pi\gamma\left[1+\left(\frac{x-x_0}{\gamma}\right)^2\right]}$	Beta	$\frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}$
Student-T	$c\left(1+\frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	LogNormal	$\frac{1}{x\sigma\sqrt{2\pi}} e^{-\frac{(\ln x-\mu)^2}{2\sigma^2}}$
Chi-squared	$\frac{1}{2^{k/2}\Gamma(k/2)} x^{k/2-1} e^{-x/2}$	Gamma	$\frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$
Logistic	$\frac{e^{-(x-\mu)/s}}{s(1+e^{-(x-\mu)/s})^2}$	Pareto	$\frac{\alpha x_m^\alpha}{x^{\alpha+1}}$
Rayleigh	$\frac{x}{\sigma^2} e^{-x^2/(2\sigma^2)}$	Weibull	$\frac{k}{\lambda}\left(\frac{x}{\lambda}\right)^{k-1} e^{-(x/\lambda)^k}$
		Gumbel-1	$\frac{1}{\beta} e^{-(z+e^{-z})}, \quad z = \frac{x-\mu}{\beta}$
		Gumbel-2	$\frac{\alpha\beta x^{-\alpha-1} e^{-(x/\beta)^{-\alpha}}}{\beta}$
		Exponentiated Power	$k\lambda x^{k-1} e^{-(x/\lambda)^k}$
Discrete Distributions			
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	Geometric	$(1-p)^{k-1}p$
Binomial	$\binom{n}{k} p^k (1-p)^{n-k}$	Negative Binomial	$\binom{k+r-1}{k}(1-p)^r p^k$
Hypergeometric	$\frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$	Bernoulli	$p^k(1-p)^{1-k}$
Discrete Uniform	$\frac{1}{n}$	Multinomial	$\frac{n!}{x_1!\cdots x_k!} p_1^{x_1}\cdots p_k^{x_k}$

Case $e = (e_1 : \texttt{float}[B; V_1] < e_2 : \texttt{float}[B; V_2])$. The Less rule was applied, so both subexpressions share the same cut set B and $\text{answers}_<(B, V_1, V_2)$ holds. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \texttt{float}[B; V_i]]\!] : \mathcal{D}[\![\texttt{float}[B; V_i]]\!]$ for $i = 1, 2$. Representation agreement gives $B_1, B_2 \neq \top$. If $B = \top$: then $\mathcal{D}[\![\texttt{float}[B; V_i]]\!] = \texttt{float}$ for both i, and both parameters are real literals satisfying the distribution's validity condition, the standard float comparison gives type $\texttt{bool} = \mathcal{D}[\![\texttt{bool}]\!]$. If $B \neq \top$: then both subexpressions are discretized to type $\texttt{fin}(|B| + 1)$, and the finite comparison $<_{\#(|B|+1)}$ direct clause emits a finite distribution over the $|B| + 1$ result cells. Its explicit target-typing premise gives type $\texttt{fin}(|B|+1)$. The formal malformed-literal clause emits divergence, which is well-typed at $\texttt{bool} = \mathcal{D}[\![\texttt{bool}]\!]$ by FinLesspolymorphically typed; an implementation runtime error is a distinct behavior and is not identified with formal divergence by this proposition.

Case $e = (e_1 : \texttt{float}[B; V_1] \leq e_2 : \texttt{float}[B; V_2])$. Analogous to the $<$ case, using $\text{answers}_\leq(B, V_1, V_2)$ and FinLessEq.

Case $e = (\texttt{uniform}(e_1 : \texttt{float}[B_1; V_1],\ e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B; \top])$. The Continuous-2 rule was applied, so $\text{recovers}(B_1, V_1)$ and $\text{recovers}(B_2, V_2)$ hold. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \texttt{float}[B_i; V_i]]\!] : \mathcal{D}[\![\texttt{float}[B_i; V_i]]\!]$ for $i = 1, 2$. We consider each sub-case of the discretization function: $e_1, e_2 \in \mathbb{R}$ and $e_1 > e_2$: then $\mathcal{D}[\![\texttt{uniform}(\ldots) : \texttt{float}[B; \top]]\!] = \texttt{diverge}$, which is well-typed at any type by Diverge.

$B = \top$: then $\mathcal{D}[\![\texttt{float}[B; \top]]\!] = \texttt{float}$, and If at least one parameter is nonliteral, the generated term has the following call-by-value shape:

$$\begin{array}{l}\texttt{let } z_1 = \mathcal{D}[\![e_1]\!] \texttt{ in} \\ \texttt{let } z_2 = \mathcal{D}[\![e_2]\!] \texttt{ in} \\ \text{dispatch}_{i=1,\ldots,n;\, j=1,\ldots,m}\left(z_1 ==_{\#(|B_1|+1)} \mathcal{D}[\![v_i]\!], z_2 ==_{\#(|B_2|+1)} \mathcal{D}[\![w_j]\!]\right)\left[\mathcal{D}[\![\texttt{uniform}(v_i, w_j) : \texttt{float}[B; \top]]\!]\right].\end{array}$$

Since $\text{recovers}(B_1, V_1)$ and $\text{recovers}(B_2, V_2)$ are preserved under discretization, the result is well-typed at $\texttt{float}$ by Continuous-2Here dispatch abbreviates the nested conditionals displayed in Figure 13.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e : \mathbf{int}}{\Gamma \vdash e \le i : \mathbf{bool}} \qquad \textsc{Pair}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 * \tau_2} \qquad \textsc{Fst}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{fst}\ e : \tau_1}$$

$$\textsc{Snd}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{snd}\ e : \tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathbf{fun}\ x\ \to\ e : \tau_1 \to \tau_2} \qquad \textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \textsc{True}\ \frac{}{\Gamma \vdash \mathrm{true} : \mathbf{bool}}$$

$$\textsc{False}\ \frac{}{\Gamma \vdash \mathrm{false} : \mathbf{bool}} \qquad \textsc{And}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \mathbf{bool}} \qquad \textsc{Or}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \mathbf{bool}}$$

$$\textsc{Not}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathrm{not}\ e : \mathbf{bool}} \qquad \textsc{Unit}\ \frac{}{\Gamma \vdash () : \mathbf{unit}} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Seq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2}$$

$$\textsc{Fix}\ \frac{\Gamma, f : \tau_1 \to \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathrm{fix}\ f\ x := e : \tau_1 \to \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \mathrm{nil} : \mathbf{list}(\tau)} \qquad \textsc{Cons}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \mathbf{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \mathbf{list}(\tau)}$$

$$\textsc{Match}\ \frac{\Gamma \vdash e : \mathbf{list}(\tau_1) \qquad \Gamma \vdash e_1 : \tau_2 \qquad \Gamma, h : \tau_1, t : \mathbf{list}(\tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \mathrm{match}\ e\ \mathrm{with\ nil} \to e_1 \mid h :: t \to e_2\ \mathrm{end} : \tau_2} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

$B \neq \top$ and $e_1, e_2 \in \mathbb{R}$: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{fin}(|B| + 1)$, and where $p_k = \mathrm{CDF}(b_{k+1}) - \mathrm{CDF}(b_k)$. Each $p_k \in [0, 1]$ and $\sum_k p_k = 1$, so this is well-typed at Writing $\mathcal{D}[\![\Gamma]\!]$ for the pointwise transformation of the types in Γ, assume the recursive premises

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i]\!] : \mathbf{fin}(|B_i| + 1) \quad (i \in \{1, 2\})$$

and the stated target typing rules for let-binding, finite equality, and conditionals. Each $\mathcal{D}[\![v_i]\!]$ has type $\mathbf{fin}(|B_1| + 1)$ and each $\mathcal{D}[\![w_j]\!]$ has type $\mathbf{fin}(|B_2| + 1)$, so every dispatch guard is Boolean. Assume also that the local constant-parameter rule produces each valid arm at type $\mathbf{fin}(|B| + 1)$ by Discrete.

$B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, and e_1 or e_2 is a variable: the algorithm generates conditionals over all pairs (v_i, w_j), each branch dispatching to a constant parameter in **uniform**. By the previous sub-case, each branch $\mathcal{D}[\![\mathbf{uniform}(v_i, w_j) : \mathbf{float}[B; \top]]\!]$ is well-typed at $\mathbf{fin}(|B|+1) = \mathcal{D}[\![\mathbf{float}[B; \top]]\!]$. The conditional structure is well-typed by If (as in the **if** case above), so the full expression is well-typed at $\mathcal{D}[\![\mathbf{float}[B; \top]]\!]$.

Case e is any other language construct (functions, pairs, lists, *etc.*). By definition of $\mathcal{D}[\![\cdot]\!]$, the discretization recurses structurally over e and that its formal invalid-parameter arm is typed as divergence. Consequently

every conditional branch, and τ. That is, $\mathcal{D}[\![e : \tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each subexpression and subtype of $e : \tau$, and $\mathcal{D}[\![\tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each component type of τ. The result then follows immediately by applying the inductive hypothesis to each subterm, analogous to the above cases. hence the complete generated dispatch term, has type $\mathbf{fin}(|B| + 1)$.

In every case, the typing derivation is preserved under $\mathcal{D}[\![\cdot]\!]$. Therefore, This establishes representation agreement for the continuous-distribution case only. It does not establish the former universal "Discretization Is Well-Typed" statement. A universal type-preservation result still needs independent lemmas for Sub-Type, the nonempty-dispatch invariant, the literal generated target syntax, and the coupling obligations used later in the soundness argument. □

F Soundness Proof

F.1 Measure Theory Preliminaries

We first give a brief overview of basic measure-theoretic terms and lemmas, and refer the reader to standard textbooks for further background [43, 44].

Topological space. A topological space is a pair $(X, \mathcal{T})$ where $\mathcal{T}$ is a collection of subsets of X, called open sets, such that $\emptyset, X \in \mathcal{T}$, and $\mathcal{T}$ is closed under arbitrary unions and finite intersections.

σ-algebra. A σ-algebra on a set X is a collection Σ of subsets of X that includes X itself and is closed under complement and under countable unions. By implication, a σ-algebra includes the empty set $\emptyset$ and is closed under countable intersections. Given a topological space $(X, \mathcal{T})$, the Borel σ-algebra $\mathcal{B}(X)$ is the smallest σ-algebra containing all open sets in $\mathcal{T}$.

Measurable space. A measurable space is a pair (X, Σ) consisting of a set X and a σ-algebra Σ on X. When Σ is the Borel σ-algebra on a topological space X, we call (X, Σ) a Borel measurable space.

Measure. Let (X, Σ) be a measurable space. A function $\mu : \Sigma \to [0, \infty]$ is called a measure if it satisfies the following properties: nonnegativity, $\mu(\emptyset) = 0$, and countable additivity. When Σ is a Borel σ-algebra, μ is called a Borel measure.

Measure space. The triple (X, Σ, μ) is called a measure space. When Σ is the Borel σ-algebra and μ is a Borel measure, (X, Σ, μ) is called a Borel measure space.

Subprobability measure. Let (X, Σ, μ) be a measure space. When $\mu(X) \leq 1$, μ is called a (Borel) subprobability measure. When $\mu(X) = 1$, μ is called a (Borel) probability measure and (X, Σ, μ) a (Borel) probability space.

Product measurable space. The product of two measurable spaces (X_1, Σ_1) and (X_2, Σ_2) is also a measurable space, namely $(X_1 \times X_2, \Sigma_1 \times \Sigma_2)$, where $\Sigma_1 \times \Sigma_2$ is the σ-algebra generated by sets of the form $A_1 \times A_2$ with $A_1 \in \Sigma_1$ and $A_2 \in \Sigma_2$.

Generated σ-algebra. Let X be a set and let $\mathcal{G} \subseteq \mathcal{P}(X)$ be a collection of subsets of X. The σ-algebra generated by $\mathcal{G}$, denoted $\sigma(\mathcal{G})$, is the smallest σ-algebra on X that contains $\mathcal{G}$. Equivalently, it is the intersection of all σ-algebras on X that contain $\mathcal{G}$.

Subprobability distribution. Let (X, Σ) be a measurable space. We write $\mathcal{D}(X)$ for the set of all subprobability measures on (X, Σ).

Dirac measure. Let (X, Σ) be a measurable space and let $x \in X$. The *Dirac measure* at x is the probability measure

$$\delta_x : \Sigma \to [0, 1]$$

defined by

$$\delta_x(A) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A. \end{cases}$$